

pWin.ai Opportunities Quick Start

`pWin.ai Opportunities` helps you bootstrap a workspace from a company website, scan federal opportunities, turn promising items into capture briefs, and improve future matching with plain-English feedback.

This version reflects the current shipped behavior:

- official-source scan and capture run on `SAM.gov` plus `USAspending.gov`
- scan can use OpenAI reasoning to read opportunity text more like a human reviewer
- `GovTribe MCP` is now an optional commercial-intelligence sidecar for scan and capture
- capture can run from a stable ID, from local files, or from both together
- capture evidence now includes normalized cross-source fields such as incumbent, vehicle, related procurements, teaming posture, and source conflicts when that evidence is available

Minimum Requirements

- One working host: `OpenAI Codex`, `OpenClaw`, or `Claude Code`
- `python3` available in your shell
- A workspace folder where the skill can write artifacts
- A `SAM.gov` API key exposed as `SAM_API_KEY`
- Internet access
- GitHub access to this repo

Strongly recommended:

- An OpenAI API key exposed as `OPENAI_API_KEY`
- A reasoning-capable OpenAI model for your host

Optional:

- A `GovTribe MCP` API key exposed as `GOVTRIBE_MCP_API_KEY`

Base scan and capture work with `SAM_API_KEY` alone. `OPENAI_API_KEY` enables only the shipped semantic reasoning path. `GovTribe MCP` uses `GOVTRIBE_MCP_API_KEY` directly, is optional, and is disabled by default per workspace.

If your GitHub setup uses HTTPS instead of SSH, swap the clone URLs below to the HTTPS form you normally use.

1. Install the Skill

OpenAI Codex

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git
"$HOME/.codex/skills/pwin-ai-opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/.codex/skills/pwin-ai-opportunities"
```

If Codex does not pick it up immediately, restart Codex or refresh skill discovery.

OpenClaw

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git
"$HOME/.openclaw/skills/pwin-ai-opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/.openclaw/skills/pwin-ai-opportunities"
```

Claude Code

```
git clone git@github.com:vlele/pwin-ai-opportunities-engine.git "$HOME/src/pwin-ai-
opportunities"
export PWIN_AI_OPPTS_ROOT="$HOME/src/pwin-ai-opportunities"
mkdir -p "$HOME/.claude/commands/pwin-ai-opportunities"
cp -R "$PWIN_AI_OPPTS_ROOT/claude-code/.claude/commands/." "$HOME/.claude/commands/pwin-
ai-opportunities/"
```

Then copy or merge `claude-code/CLAUDE.md` into the Claude Code project context you normally use.

2. Add Your Keys

Required minimum:

```
export SAM_API_KEY="your-sam-gov-key"
```

Recommended for the shipped reasoning path:

```
export OPENAI_API_KEY="your-openai-key"
export OPENAI_MODEL="gpt-5.4-mini"
```

Optional for the `GovTribe MCP` sidecar:

```
export GOVTRIBE_MCP_API_KEY="your-govtribe-mcp-key"
export GOVTRIBE_MCP_URL="https://govtribe.com/mcp"
export GOVTRIBE_MCP_TIMEOUT_SECONDS=90
```

Notes:

- If `OPENAI_MODEL` is not set, the shipped scripts default to `gpt-5.4-mini`.
- If `GovTribe MCP` is enabled in a workspace but `GOVTRIBE_MCP_API_KEY` is missing, scan and capture still run, but the commercial sidecar reports `not_configured`.
- `GovWin IQ` is only a credential scaffold today, not a live runtime connector.

3. Pick a Workspace

Create or open one folder per company:

```
mkdir -p "$HOME/work/acme-capture"  
cd "$HOME/work/acme-capture"
```

This skill writes its outputs into `procurement/` inside that workspace.

4. Bootstrap First, Then Scan

Use this rule of thumb:

- if this is a brand-new workspace, bootstrap first
- if `procurement/vendor-profile.json` does not contain confirmed or candidate NAICS, bootstrap first
- if a scan returns `no_naics`, treat that as a bootstrap prompt, not a successful first scan

A workspace is ready for its first real scan when all of these are true:

- `procurement/vendor-profile.json` exists
- the vendor profile contains confirmed or candidate NAICS
- `procurement/STARTER_PROFILE.md` exists for review before the first scan

5. Bootstrap the Workspace from the Company URL

In OpenClaw or Codex, ask:

```
Use pwin-ai-opportunities and bootstrap this workspace from https://example.com.
```

If you already know the NAICS codes, say:

```
Use pwin-ai-opportunities and bootstrap this workspace from https://example.com  
with confirmed NAICS 541511 and 541512.
```

If those NAICS are still tentative, say:

```
Use pwin-ai-opportunities and bootstrap this workspace from https://example.com
and treat NAICS 541511 and 541512 as candidates.
```

If you are already looking at a GovTribe vendor profile and `GOVTRIBE_MCP_API_KEY` is configured, you can bootstrap from GovTribe instead:

```
Use pwin-ai-opportunities and bootstrap this workspace from
https://govtribe.com/vendors/demogov-services-demo1.
```

In Claude Code, use:

```
/pwin-bootstrap workspace path $PWD and company URL https://example.com
/pwin-bootstrap workspace path $PWD and company URL https://example.com with confirmed
NAICS 541511 and 541512
/pwin-bootstrap workspace path $PWD and company URL https://example.com with candidate
NAICS 541511 and 541512
```

If you want the direct underlying CLI path on any host, run:

```
python3 "$PWIN_AI_OPPS_ROOT/scripts/bootstrap/bootstrap_workspace.py" \
  --workspace "$PWD" \
  --company-url "https://example.com"
```

For GovTribe vendor lookup:

```
python3 "$PWIN_AI_OPPS_ROOT/scripts/bootstrap/bootstrap_workspace.py" \
  --workspace "$PWD" \
  --vendor-lookup "DemoGov Services, LLC"
```

That creates or updates:

- `procurement/vendor-profile.json`
- `procurement/preferences.json`
- `procurement/source-registry.json`
- `procurement/STARTER_PROFILE.md`
- `MEMORY.md`

Bootstrap does more than create empty files. It immediately pre-populates `procurement/vendor-profile.json` with:

- bootstrap metadata and a `needs_user_confirmation` status

- `company.website` plus an inferred company name and summary
- inferred `core_competencies`, `other_taxonomy_tags.keywords`, and a starter `fit_narrative`
- confirmed `naics.confirmed` values when you provide confirmed codes
- candidate `naics.candidates` values from tentative input plus website inference
- candidate buyer notes, provenance facts, and a reminder that website-derived facts are still provisional

GovTribe bootstrap can also pre-populate UEI, GovTribe profile URL, NAICS, certifications, contract vehicles, award and buyer signals, and AI summary text. Those facts are labeled as GovTribe subscription-derived commercial intelligence and remain provisional until confirmed.

It also seeds `procurement/preferences.json` to exclude `grants` by default, prefer `contracts`, `subcontracts`, and `forecasts`, and reuse the inferred capability keywords. If `procurement/vendor-profile.json` already exists, bootstrap merges into it instead of resetting the file.

Before the first real scan, review `procurement/STARTER_PROFILE.md` and confirm or correct the provisional fields.

6. Optional: Enable GovTribe in This Workspace

You can skip this section and stay official-source only.

If you want GovTribe MCP enrichment in this specific workspace, flip it on in `procurement/source-registry.json`:

```
python3 - <<'PY'
import json
from pathlib import Path

path = Path("procurement/source-registry.json")
data = json.loads(path.read_text())
for source in data.get("sources", []):
    if source.get("id") == "govtribe_mcp_commercial_intel":
        source["enabled"] = True
path.write_text(json.dumps(data, indent=2) + "\n")
PY
```

What GovTribe MCP adds when it is both enabled and configured:

- optional scan-time commercial-intelligence matching
- optional capture-time incumbent, vehicle, related-procurement, and teaming clues
- normalized cross-source evidence with explicit conflict reporting when official and commercial signals disagree

`SAM.gov` remains the recommended official retrieval path. If a workspace intentionally needs GovTribe-only scan retrieval when `SAM_API_KEY` is unavailable, also set the explicit provider option:

```
python3 - <<'PY'
import json
from pathlib import Path

path = Path("procurement/source-registry.json")
data = json.loads(path.read_text())
for source in data.get("sources", []):
    if source.get("id") == "govtribe_mcp_commercial_intel":
        source["enabled"] = True
        source.setdefault("provider_options", {})[
            "allow_scan_retrieval_without_sam"
        ] = True
path.write_text(json.dumps(data, indent=2) + "\n")
PY
```

This is a commercial tier-4 alternate retrieval path. It runs only when SAM.gov is disabled or cannot run because `SAM_API_KEY` is missing; it does not replace SAM.gov when SAM is configured.

7. Run the First Scan

Once the workspace is bootstrapped, ask:

Use `pwin-ai-opportunities` and run a federal opportunity scan for the next 30 to 45 days in this workspace.

In Claude Code, use:

```
/pwin-scan workspace path $PWD and horizon 30-45
```

Expected behavior:

- if `SAM_API_KEY` is missing, the scan reports `missing_api_key`
- if GovTribe-only scan retrieval is explicitly opted in and `GOVTRIBE_MCP_API_KEY` is configured, the scan can still produce commercial tier-4 opportunity records without `SAM_API_KEY`
- if the workspace still has no usable NAICS, the scan reports `no_naics` and returns a bootstrap recommendation with the next command to run
- if `OPENAI_API_KEY` is present, the scan can add semantic fit reasoning and a `semantic_audit` block
- if GovTribe MCP is enabled and configured, the scan can add commercial source statuses plus `cross_source_evidence_notes`
- if the scan finds matches, you will get a dated digest with stable IDs like `A1`, `W2`, or `E1`

If you hit `no_naics`, rerun bootstrap before you try another scan:

```
python3 "$PWD/ai_opps_root/scripts/bootstrap/bootstrap_workspace.py" \
  --workspace "$PWD" \
  --company-url "https://example.com"
```

Key scan artifacts:

- `procurement/opportunities/YYYY-MM-DD.json`
- `procurement/explanations/YYYY-MM-DD.json`
- `procurement/digest-entry-map/YYYY-MM-DD.json`
- `procurement/digests/YYYY-MM-DD.md`

8. Read the Digest

Ask:

```
Use pwin-ai-opportunities and show the latest digest.
```

In Claude Code, use:

```
/pwin-show-digest workspace path $PWD and date latest
```

The digest is where you pick stable IDs for tracked capture.

9. Run Capture Research

You now have three supported capture paths:

- tracked capture from a digest stable ID such as `A1`
- direct local-file capture with one or more `--file` inputs plus metadata
- hybrid capture that starts from a stable ID and adds local files

Tracked capture from a stable ID

Ask:

```
Use pwin-ai-opportunities and research A1 with full capture depth.
```

In Claude Code, use:

```
/pwin-research workspace path $PWD and entry A1 with full capture depth
```

Direct local-file capture

```
python3 "$SPWIN_AI_OPFS_ROOT/scripts/capture/run_capture_research.py" \
  --workspace "$PWD" \
  --file "/absolute/path/to/PWS.pdf" \
  --file "/absolute/path/to/QA.docx" \
  --title "Example opportunity" \
  --buyer "Department of Example" \
  --summary "Direct local-file capture from downloaded solicitation artifacts." \
  --solicitation-number "ABC123" \
  --depth full_360
```

Optional metadata for direct local-file capture includes `--url` and `--notice-id` when you have them.

Hybrid capture

```
python3 "$SPWIN_AI_OPFS_ROOT/scripts/capture/run_capture_research.py" \
  --workspace "$PWD" \
  --entry "A1" \
  --file "/absolute/path/to/amendment.pdf" \
  --depth full_360
```

All three capture modes write fresh artifacts under:

- `procurement/capture-briefs/`
- `procurement/capture-evidence/`

If `GovTribe MCP` is enabled and configured, capture evidence may include:

- `commercial_intel.source_statuses`
- `commercial_intel.matches`
- `cross_source_evidence`
- normalized fields for incumbent, vehicle, recompetete clues, related procurements, contract value or ceiling, teaming posture, next questions, and conflicts

10. Give Feedback

Use plain-English feedback such as:

```
never show grants
like A1
dislike W2 because too small
more like A1
```

In Claude Code, use:

```
/pwin-feedback workspace path $PWD and text never show grants  
/pwin-feedback workspace path $PWD and text like A1
```

That feedback is logged to `procurement/feedback-events.jsonl` and applied to future scans.

What Gets Created in Your Workspace

- `procurement/vendor-profile.json`
- `procurement/preferences.json`
- `procurement/source-registry.json`
- `procurement/STARTER_PROFILE.md`
- `procurement/opportunities/YYYY-MM-DD.json`
- `procurement/explanations/YYYY-MM-DD.json`
- `procurement/digest-entry-map/YYYY-MM-DD.json`
- `procurement/digests/YYYY-MM-DD.md`
- `procurement/reports/YYYY-MM-DD.md`
- `procurement/capture-briefs/...`
- `procurement/capture-evidence/...`
- `procurement/feedback-events.jsonl`
- `MEMORY.md`

Troubleshooting

- `missing_api_key` `SAM_API_KEY` is not exported in the shell running the scan.
- `no_naics` The workspace still does not have confirmed or candidate `NAICS` after bootstrap or manual edits.
- `govtribe_mcp_commercial_intel: not_configured` The workspace has GovTribe enabled, but the runtime shell is missing `GOVTRIBE_MCP_API_KEY`.
- `govtribe_mcp_commercial_intel: tool_contract_unavailable` GovTribe is reachable, but the available MCP tools or schemas do not match the provider contract. The official-source path should still complete.
- GovTribe `error` or `IncompleteRead` Rerun the scan or capture. The official-source path should still complete even when the commercial sidecar has a transient failure.
- `HTTP 429` from `SAM.gov` Your key is wired correctly, but the API quota is throttled. Wait until the `nextAccessTime` returned by `SAM.gov` and rerun.

Short Version

1. Clone the repo with your normal authenticated GitHub method.
2. Export `SAM_API_KEY`.
3. Export `OPENAI_API_KEY` if you want the shipped reasoning path.
4. Export `GOVTRIBE_MCP_API_KEY` only if you want the optional GovTribe sidecar.
5. Create a workspace.
6. Bootstrap it from the company website.
7. Review the starter profile and the populated `vendor-profile.json`.
8. Optionally enable GovTribe in `procurement/source-registry.json`.
9. Run the federal-only scan.
0. Read the digest, research a stable ID or local files, and give feedback.

Direct Script Equivalents

These are helpful for debugging or CI. In normal OpenClaw and Codex usage, prefer the chat prompts above. In Claude Code, prefer the `/pwin-*` command pack.

```
python3 "SPWIN_AI_OPFS_ROOT/scripts/bootstrap/bootstrap_workspace.py" --workspace "$PWD"
--company-url "https://example.com"
python3 "SPWIN_AI_OPFS_ROOT/scripts/scan/run_scan.py" --workspace "$PWD" --horizon "30-
45" --federal-only
python3 "SPWIN_AI_OPFS_ROOT/scripts/show/show_digest.py" --workspace "$PWD" --date
latest
python3 "SPWIN_AI_OPFS_ROOT/scripts/capture/run_capture_research.py" --workspace "$PWD"
--entry "A1" --depth full_360
python3 "SPWIN_AI_OPFS_ROOT/scripts/capture/run_capture_research.py" --workspace "$PWD"
--file "/absolute/path/to/PWS.pdf" --file "/absolute/path/to/QA.docx" --title
"Example opportunity" --buyer "Department of Example" --summary "Direct local-
file capture from downloaded solicitation artifacts." --solicitation-number
"ABC123" --depth full_360
python3 "SPWIN_AI_OPFS_ROOT/scripts/capture/run_capture_research.py" --workspace "$PWD"
--entry "A1" --file "/absolute/path/to/amendment.pdf" --depth full_360
python3 "SPWIN_AI_OPFS_ROOT/scripts/feedback/apply_feedback.py" --workspace "$PWD" --
text "never show grants"
```